Genie Code Workshop Labs

Overview

Genie Code is an autonomous AI partner purpose-built for data work in Databricks. Unlike other AI assistants, Genie Code is deeply integrated with Unity Catalog, allowing it to understand your complete data landscape, including tables, columns, and lineage. This contextual awareness enables Genie Code to accelerate complex data workflows while autonomously adapting to your specific data and governance model. Genie Code is designed for data teams to use every day, from experimentation and model development to production pipelines and BI dashboards. It is available throughout the Databricks Lakehouse, and chat threads persist as you navigate between pages.

These hands-on labs will guide you through Genie Code's core capabilities using a Vocareum hosted workspace. Vocareum provides infrastructure for labs just like this.

What you will cover

- **Lab 1: Getting Started - Modes and Navigation** - Get familiar with Genie Code basics.
- **Lab 2: Exploratory Data Analysis with Agent Mode** - Let Agent Mode autonomously plan and execute a full exploratory data analysis (EDA).
- **Lab 3: SQL Query Optimisation and Debugging** - Format, document, optimise, and repair broken SQL.
- **Lab 4: Build a Data Transformation Workflow** - Construct a full medallion ETL pipeline, extending it iteratively via follow-up prompts.
- **Lab 5: Data Analysis and Visualisation** - Create an AI/BI Dashboard from the Lab 4 tables - all driven by natural language.
- **Lab 6: Price Prediction with Machine Learning** - Build, train, and evaluate an end-to-end ML model that predicts property price.
- **Lab 7: AI Functions** - Using various ai_functions in Databricks such as ai_classify, ai_summarize and ai_query.
- **Lab 8: Genie Spaces** - Build a Genie Space to support natural language analysis.

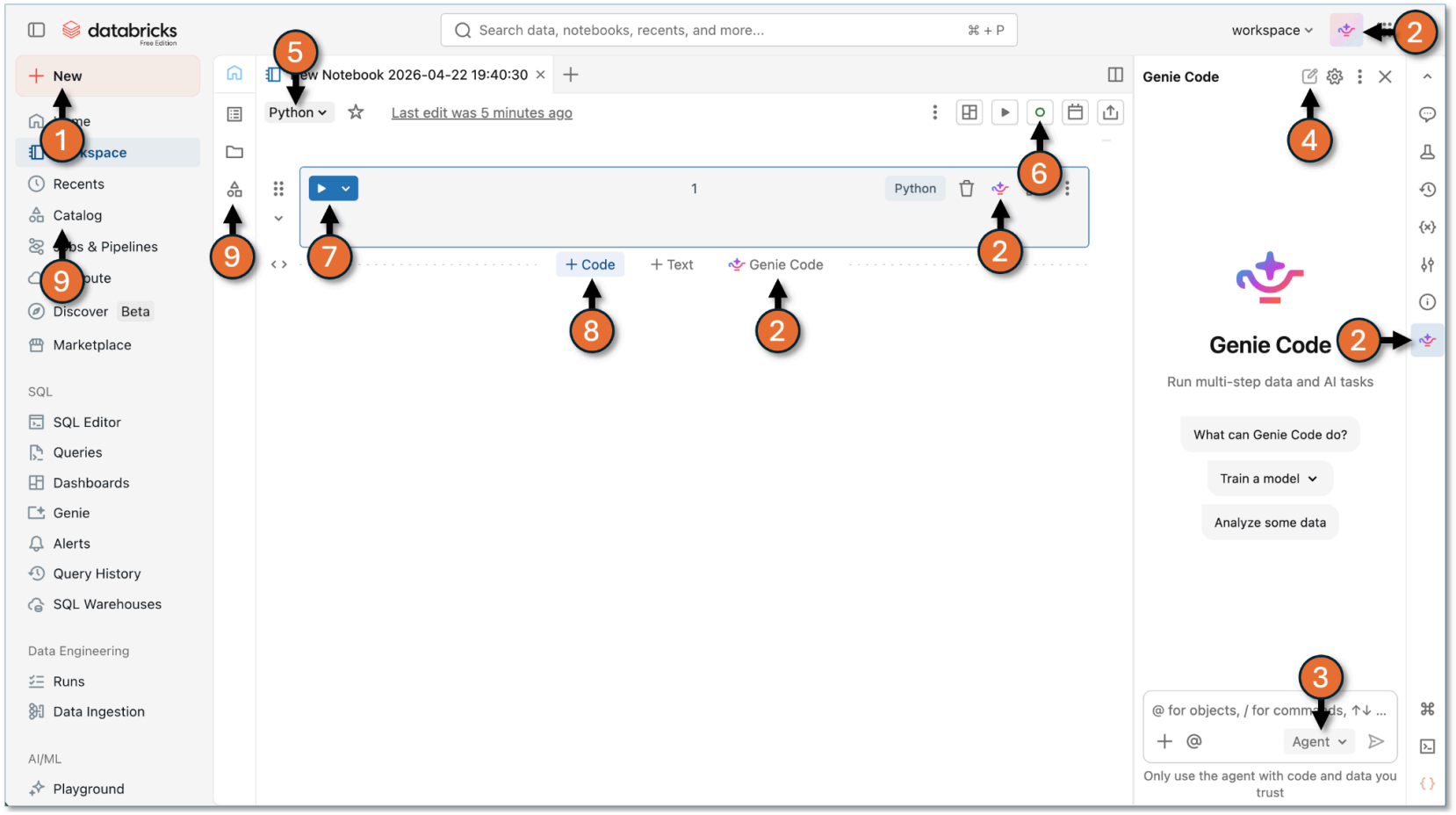
**** IMPORTANT ****

- Labs 1–4, 7 and 8 are standalone with no pre-requisite dependencies on any other lab. Labs 5 and 6 depend on the tables created in Lab 4.
- You may not finish every lab today - you can pivot to Free Edition and rerun these labs at your own pace.
- Instructions in earlier labs are more verbose than later labs - the assumption is you don’t need every specific detail as you move through labs.
- You can complete these labs without prior Databricks experience, but a basic understanding will help you better appreciate Genie Code’s value.

Tips

- **Your Vocareum lab** is a shared environment, so it may be slower than production Databricks.
- **Use** @<table-name> **to point Genie Code at a specific table**. When you paste a fully-qualified name from this doc (e.g. @samples.tpch.orders), click the pasted text and select the table from the picker. Shortcut: type just @orders and pick the right one from the list.
- **Approvals and controls** - Genie Code asks for approval before running code or updating pipelines. Click "**Allow**", "**Decline**", "**Continue**", or "**Reject**" as appropriate. You can also select "**Allow in this thread**" or "**Always allow**" to reduce prompts. In these labs, we recommend you click "**Allow**" (one step at a time) so you can observe what is happening. To stop Genie Code mid-task, click the red stop button ■.
- **Genie Code is non-deterministic** - your results may vary. So:
    - Always **review generated code** before executing. Genie Code is powerful but can make mistakes.
    - **Be specific** - more context = better output. Include column names, expected formats, and desired visualisation types.
    - **Try both modes** - switch between Agent Mode (for doing) and Chat Mode (for understanding) to see how they differ.
    - **Most importantly, play** - write your own prompts to explore. If you get errors, work with Genie Code to resolve.
- Some of the labs create reference tables in your schema. To identify your schema, in the left menu click on "**Catalog**", "**dbacademy**", "**labuserXXXXXXXX_XXXXXXXXXX**". Any time you see "**<your schema>**" in the rest of this document, make sure you change that to your lab schema name.
- Genie Code is a **rapidly evolving** technology. The experience and features may change (for the better) when you next try these labs.
- **Resources:**
    - Genie Code Documentation (AWS, Azure, GCP)
    - Introducing Genie Code - launch announcement and feature overview
    - Get to Know Genie Code - demo covering pipelines, dashboards, and Agent Mode
    - Databricks YouTube Channel (filtered by "Genie Code") - tutorials and demos
    - Databricks Genie Code: Full Practice Guide and Examples
    - From Genie to Genie Code: Agentic Data Work on Databricks

The Databricks Workspace UI (after clicking “+ New”, “Notebook”)



1. Click to create a new Notebook / ETL Pipeline / etc.
2. Bring up the Genie Code panel
3. Toggle Genie Code between **Chat** and **Agent** modes
4. Start a new chat
5. Toggle Notebook default language between **Python** and **SQL**
6. Select compute type (**Serverless** or **Serverless Starter Warehouse**)
7. Run Cell
8. Click “**+ Code**” to add an empty cell below the current cell
9. Unity Catalog - explore data related objects

Lab 1: Getting Started - Setup, Modes and Navigation

Objective: Setup your Databricks Vocareum workspace; Learn to interact with Genie Code; Explore the UI; Use @ table references.

1. Go to https://dbfe.vocareum.com.
2. If you have used Vocareum before, click on "**Log in**" - enter your email address, then click on "**Send one-time code to log in**". Retrieve the code from your email and use it to log in.
3. If this is your first time using Vocareum or unsure, click on "**Sign Up** - enter your name and your preferred email address. Retrieve the code from your email and use it to log in.
4. You should see multiple tiles for different workshops we have run. Search for the tile "**Auckland Learning Festival**" (it may be at the bottom). Click on "**Enroll now**".
5. When prompted for a code enter "**auckland-learning-festival**"
6. After approximately 1-2 minutes your lab environment should launch (some may take up to 5 minutes).
7. Click on "**Databricks Workspace**" to open the Databricks Workspace UI. Note you can shut your lab down by clicking on "**End Lab**" at any time. Note your lab can run for a maximum of 4 hours in one go and 12 hours in total.
8. In the Databricks Workspace UI click on "**+ New**" in the main menu on the left, then select "**Notebook**".
9. At the top left you should see "**Python**". This is the notebook's default language. Click on this and change it to "**SQL**".
10. Open the Genie Code pane by clicking the **Genie Code** icon in the upper-right corner .
11. Ensure "**Agent**" (versus "**Chat**") is selected at the bottom right (this is the default).
12. Start this and each subsequent lab with a fresh chat using the "New chat" button at the top of the Genie Code pane. .

    Within a lab, build on the same conversation for richer context.
13. In the chat pane, type the following prompt:

    > What tables are available in the samples.tpch schema? Describe each table and how they relate to each other.
14. Genie may ask to run code, potentially multiple times; Recommend you leave the default "**Ask every time**" and click "**Allow**".
15. Genie may also create new cells in the notebook (e.g. a Markdown Cell). If Genie Code does this, click "**Accept**" within the Notebook.
16. Review the final response. Genie Code uses Unity Catalog metadata to describe the tables and their relationships.

17. Use the **@** reference to ask a specific question about a specific table (💡remember @ references from the Tips section above):

    > How many columns does @samples.tpch.orders have? What does each column represent?

18. Next, ask Genie Code to generate a simple query:

    > Write a SQL query that shows the top 10 customers by total order value from @samples.tpch.orders and @samples.tpch.customer

19. Genie Code should generate the SQL in a new cell in the notebook. When prompted to run the cell, click "**Allow**".

    You may also see "**Run suggested**", "**Reject**", and "**Accept**" buttons in the cell — ignore them and drive execution via Genie Code's "**Allow**".

20. Try the **/explain** command. Either:
    - click the Genie icon and type /explain
    - in your cell, press "**Cmd+I**" (Mac) or "**Ctrl+I**" (Windows) and type /explain
    - in the Genie Code pane type explain this query line by line

21. Note you can also discover related data by typing something like (in the Genie Code pane):

    > /findTables related to customer transactions and sales

22. Switch Genie Code to "**Chat**" mode (bottom right of the Genie Code panel where it should currently show "**Agent**"). Enter the following question:

    > What is a window function in Spark? How does it differ from a regular GROUP BY? When would I use one over the other?

23. Follow up with:

    > What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()? Show me an example using samples.tpch.orders where the difference matters - where the three functions produce different results.

24. If example code appears, use the '"**<<**" button at the top of the snippet to insert it into a new cell. If you don't see this option you should see the "**Copy code**" button - use this to copy the code into a new cell.

    To create a new cell, move the mouse below the last cell until "- - - **+Code** **+Text** **Genie Code** - - -" pops up - click on "**+Code**".

    Once the code has been entered into the new cell, run the sql statement (hit the "▶" run cell button top left). Verify you can see where the three functions diverge.

Lab 2: Exploratory Data Analysis with Agent Mode

Objective: Use Genie Code to perform a full exploratory data analysis on a dataset, demonstrating Genie Code's ability to plan, execute, and iterate autonomously.

1. "**+ New**" → "**Notebook**" ✅, Default language "**Python**" ✅, Genie Code pane open ✅, "**New Chat**" ✅, **Agent** Mode ✅.
2. Enter the following prompt:

   > Perform an exploratory data analysis on @samples.nyctaxi.trips. Include: row count, schema overview, summary statistics for numeric columns, distribution of trip distances, fare amount trends by hour of day, and identify any data quality issues. Use Python with matplotlib for visualisations.

3. Watch as Genie Code plans the analysis steps / creates multiple notebook cells.
4. Once the cells have been created, run each cell within the Notebook (as opposed to Genie Code) to observe what happens (i.e. click "**Run cell**" / "►" run cell button top left). For any cell you want to understand better, select the code, press **Cmd+I**, and type:

   > /explain

5. If you encounter any errors, follow the prompts to use Genie to diagnose the error and rerun revised code (this may be easier to do by clicking "**Accept**" in Genie Code and have it run the cells again for you - it should handle the error and retry, repeating if necessary until it works).
6. If you haven't already, in the Genie Code pane click **Allow** to run all cells. It will re-execute the same cells which is ok before moving on to summarise its findings.
7. Now ask a follow-up question in the Genie Code pane:

   > Based on the EDA, what are the most interesting patterns? Are there any outliers worth investigating?

8. Let Agent Mode dig deeper into the patterns it identified (which includes additional code cells - run these via the Genie Code pane when prompted).
9. Review the deeper analysis.

Lab 3: SQL Query Optimisation and Debugging

Objective: Use Genie Code's /optimize, and diagnose error features to debug and improve SQL queries.

1. "**+ New**" → "**Notebook**" ✅, Default language "**SQL**" ✅, Genie Code pane open ✅, "**New Chat**" ✅, **Agent** Mode ✅.
2. Paste the following SQL statement into the empty cell at the top of the Notebook - this is intentionally suboptimal and messy:

```
SELECT c.c_name, sum(l.l_extendedprice * (1 - l.l_discount)) as revenue
FROM samples.tpch.customer c, samples.tpch.orders o, samples.tpch.lineitem l
WHERE c.c_custkey = o.o_custkey and o.o_orderkey = l.l_orderkey
AND o.o_orderdate >= '1994-01-01' and o.o_orderdate < '1995-01-01'
GROUP BY c.c_name ORDER BY revenue DESC
```

3. Using the Genie Code pane, type:

   > /prettify

4. You should see the original version highlighted in red and the new version highlighted in green. Click "**Accept"** to accept the formatted version. The query should now be much more readable.
5. This next step does not happen in the Genie Code pane! Instead, highlight the code in the cell, and either click the kebab at the top right of the cell and click on the Genie icon or press "**Cmd+I**" (Mac) or "**Ctrl+I**" (Windows), and type:

   > /doc

6. Accept the documentation - Genie Code will add clear inline comments explaining the query logic.
7. Run the cell (hit the "►" run cell button top left).
8. Back in the Genie Code pane, type:

   > Is the query suboptimal? If so, why?

9. Ask Genie Code to optimise it.

   > /optimize

10. Review the suggested changes - Genie Code may suggest replacing implicit joins with explicit JOIN syntax, adding appropriate filters, or restructuring the query.
11. **"Accept"** and run the adjusted code via the Genie Code pane - once done you should get an explanation why this new version is an improvement of the previous version.
12. An "**Optimize**" button also appears at the bottom right of the SQL cell. Optionally click it for a post-execution analysis, and apply any further recommendations.
13. If you ran the query before and after /optimize, try "**compare execution times**" (offered as a button, or type it into Genie Code if you don't see this button).

14. Now, create a new SQL cell with the broken query below (move the mouse below the last cell until "- - - **+Code** **+Text** **Genie Code** - - -" pops up - click on "**+Code**"):

```
SELECT c_name, total_spent
FROM samples.tpch.customer
JOIN (
  SELECT o_custkey, SUM(o_totalprice) AS total_spent
  FROM samples.tpch.orders
  WHERE o_orderdate > '1998-01-01'
  GROUPBY o_custkey
) AS order_totals
ON c_custkey = order_totals.o_custkey
```

15. Run the cell - it will fail.
16. Genie Code may go straight to "**Attempting to fix**". If not, click the "**Diagnose Error**" button that appears in the cell output.
17. Genie Code will identify the issues (e.g., `GROUPBY` should be `GROUP BY`, possible date range issues) and suggest a quick fix. Click "**Allow"** to apply and re-execute.

Lab 4: Build a Data Transformation Workflow

Objective: Use Genie Code to build a simple data transformation workflow using Lakeflow Spark Declarative Pipelines, demonstrating Genie Code's data engineering capabilities.

1. "**+ New**" → "**ETL Pipeline**" (ETL Pipeline UI opens) ✅, Genie Code pane open ✅, "**New Chat**" ✅, **Agent** Mode ✅.
2. In the ETL Pipeline UI, change the catalog and schema from `**dbacademy.default**` to `**dbacademy.<your schema>**` (top left) to avoid catalog permission issues.
3. Enter the following prompt to create the bronze tables:

> Create a pipeline with bronze streaming tables that ingest samples.wanderbricks tables properties, reviews, hosts, amenities, property_amenities, destinations. No transformations, just add table comments.

4. Genie Code may go ahead and create python or SQL files required to complete this step. Genie Code may instead provide you with a plan (e.g. you may see something like "Does this structure work for you, or would you like to modify the naming or organization?", followed by some suggested follow up prompts. Click the prompt that creates the python or SQL files (e.g. "Yes, proceed with this structure").
5. Feel free to click on any of the generated files to see the python code generated for each table.
6. When prompted "**Dry-running pipeline**", click "**Allow**". You may also get warned **"blocked execution for safety reasons"** - click "**run**" to continue.
7. Dry-running the pipeline checks the code to make sure it works as a unit. If the Dry run fails, it should diagnose what went wrong and retry.
8. Once the Dry run completes without error you will be prompted to "**Running pipeline**". Click "**Skip**".
9. Now ask Genie Code to create the silver tables:

   > Add silver tables in same schema that clean the bronze layer: cast prices to double and dates to date type, put in expectations that drop rows where base_price is null or zero, join amenities to property_amenities to get amenity names per property, aggregate reviews per property (avg scores, review count, most recent review date), standardise categoricals to lowercase. Calculate host_tenure_days from host start date.

10. Once the Dry run completes without error you will be prompted to "**Running pipeline**". As before, click "**Skip**".
11. Finally ask Genie Code to create the gold table:

    > Add a gold table gold_price_prediction_features in the same schema that joins all silver tables into one denormalised row-per-property table. Include base_price, property features, host features including host_tenure_days, destination/location features, aggregated review metrics, and pivot the top 15 most common amenities into binary flag columns. Add calculated features, price_per_person and price_per_bedroom. Add column comments.

12. Once the Dry run completes without error, when prompted with "**Running pipeline**", this time click "**Allow**" (alternatively click "**Run Pipeline"** in the top right). Make sure to select workspace.genie_demo for the pipeline table creation.
13. Click on "**Runs**" under "**Data Engineering**" in the left hand menu navigator.
14. Click on "**Pipelines**" - you should see your pipeline execution. Click on the pipeline Start time or Name to open the pipeline run and explore.
15. In the Pipeline graph, hover over gold_price_prediction_features, click the kebab (3 dots) → "**View in catalog**". In the Unity Catalog tab that opens, click "**Sample Data**" to see the rows (you may need to click "**Select Compute**" first).

Lab 5: Data Analysis and Visualisation

Objective: Use Genie Code within AI/BI Dashboards to build an interactive business dashboard from the pipeline tables created in Lab 4, demonstrating how Genie Code can generate datasets and widgets directly on a dashboard canvas.

*** **IMPORTANT** ***: This lab requires the pipeline from Lab 4 to have completed successfully.

1. "**+ New**" → "**Dashboard**" (a blank AI/BI Dashboard will open) ✅, Genie Code pane open ✅, "**New Chat**" ✅, **Agent** Mode ✅.
2. Enter the following prompt to create the first dataset and widget (click "**Allow**" if prompted, this time and each time throughout this lab):

   > Using dbacademy.**<your schema>**.gold_price_prediction_features, add a dataset and widget showing total properties, average base_price, average price_per_person, and average price_per_bedroom as a counter row across the top of the dashboard.

3. Now ask Genie Code to add a price distribution chart:

   > Add a dataset and widget showing the distribution of base_price as a histogram. Title it "Price Distribution".

4. Next, ask for a property features analysis:

   > Add a dataset and widget showing average base_price by number of bedrooms as a bar chart. Title it "Average Price by Bedrooms".

5. Add a review metrics widget:

   > Using dbacademy.**<your schema>**.silver_reviews_aggregated, add a dataset and widget showing a scatter plot of average review score vs total review count per property. Title it "Review Score vs Review Volume".

6. Add a host analysis analysis:

   > Using gold_price_prediction_features, add a dataset and widget showing the relationship between host_tenure_days and average base_price. Bucket host tenure into 6-month intervals and show average price per bucket as a bar chart. Title it "Price by Host Experience".

7. Add an amenity impact analysis:

   > Using the amenity flag columns in gold_price_prediction_features, add a dataset and widget that calculates the average base_price for properties with vs without each of the top 15 amenities. Show the price difference as a horizontal bar chart sorted by impact. Title it "Amenity Price Impact".

8. Finally, review the dashboard layout. Ask Genie Code to tidy it up:

   > Give the dashboard tab a name and rearrange the layout so the counters are at the top, the price distribution and bedrooms charts are side by side below, then the scatter plot and host experience charts side by side, and the amenity impact chart at the bottom. Use colours more effectively.

9. Click on the “**Data**” tab (top left of the Dashboard) to see all the datasets Genie Code generated to support these visualization widgets (in each case created from a SQL dataset).
10. Click “**Publish**” (top right corner), accept defaults for subsequent prompts.
11. Click “**View Published**” (top middle) to see the Dashboard as end users will see it (close Genie Code).

Lab 6: Price Prediction with Machine Learning

Objective: Use Genie Code to build, train, and evaluate a machine learning model that predicts property base_price from the feature table created in Lab 4.

Note: This lab requires the pipeline from Lab 4 to have completed successfully. The tables in workspace.genie_demo must be populated.

1. “**+ New**” → “**Notebook**” ✅, Default language “**Python**” ✅, Genie Code pane open ✅, “**New Chat**” ✅, **Agent** Mode ✅.
2. Make sure the compute you are attached to is **Serverless**, not **Serverless Starter Warehouse.**
3. Enter the following prompt:

   > Using dbacademy.**<your schema>**.gold_price_prediction_features, I want to be able to forecast base_price using property features (bedrooms, bathrooms, accommodates etc), price_per_person, price_per_bedroom, host_tenure_days, review metrics (average scores, review count), and the amenity flag columns.

4. Genie Code should generate a multi-cell ML pipeline (EDA, feature engineering, preprocessing, training - your exact cells may differ). Click **"Allow"** to run the cells via Genie Code. Genie Code may choose to run groups of cells such as the EDA steps. If an error is encountered, Genie Code should self diagnose the issue and retry.

   To understand any cell, highlight the code, click on the kebab menu and select the Genie Code icon or press **Cmd+I** (Mac) / **Ctrl+I** (Windows), and type:

   > /doc or /explain

5. Once the last cell has run, you should see output similar to the following:

   > ▾ (1) MLflow run
   >
   > Logged 1 run ↗ to an experiment ↗ in MLflow. Learn more ↗

6. Feel free to click on “**1 run**” or “**experiment**” to explore what has been created.

7. Finally test the model. Enter the following into Genie Code:

> test the model with a sample property for a 2-bedroom 2-bathroom property in San Francisco to predict the base_price

Lab 7: Analysing Text with AI SQL Functions

Objective: Use Databricks built-in AI SQL functions to classify, summarise, and generate content from unstructured text - all directly in SQL, with no model serving or MLflow setup required.

Note: AI SQL functions run on Databricks-managed foundation model endpoints and are available on Serverless. Keep LIMIT clauses modest while you are exploring - these functions can consume tokens fast. That said, the dataset we are using in this lab only has 204 rows, so you should be OK.

Note: We are only using 3 of the ai functions available - see the documentation for the full list.

1. "**+ New**" → "**Notebook**" ✅, Default language "**SQL**" ✅, Genie Code pane open ✅, "**New Chat**" ✅, **Agent** Mode ✅.
2. Start with a quick reconnaissance of the dataset:

> What does @samples.bakehouse.media_customer_reviews contain? Show me the schema and 5 sample rows with the full review text.

3. Genie Code may do the sampling in Genie Code or it may do it in the Notebook, in which case accept and run the generated query and run it. Confirm the result set has the free-text "**review**" column - everything that follows uses it.
4. First we will use **ai_classify**. Create a new SQL cell with the query below:

```
SELECT
  ai_classify(
    'The sourdough was incredible, crispy crust and perfect crumb. Coffee was average.',
    ARRAY('positive', 'negative', 'mixed', 'neutral')
  ) AS sentiment;
```

5. Run the cell (hit the "►" run cell button top left). It should come back with "**mixed**".
6. Apply the function at scale. Ask Genie Code:

> Classify every row in @samples.bakehouse.media_customer_reviews as positive, negative, mixed, or neutral using ai_classify, and return the count per label ordered by the most common.

7. Accept and run the generated SQL. You now have sentiment distribution across the whole review set - no model deployment, just SQL.

8. Now we will use **ai_summarize**. Produce one short summary per sentiment label in a single query. Create a new SQL cell with the query below:

```
SELECT
  ai_classify(review, ARRAY('positive', 'negative', 'mixed', 'neutral')) AS sentiment,
  ai_summarize(array_join(collect_list(review), ' | '), 40) AS summary
 FROM samples.bakehouse.media_customer_reviews
GROUP BY 1;
```

9. You should get one row per sentiment label, each with a summary distilled from all the reviews in that group. The second argument to ai_summarize **(40)** is the maximum length of the summary in words - change it to **15** and re-run to see the effect on terseness.
10. Next we will use **ai_query**. This general-purpose function calls a foundation model with your own prompt (in this case **databricks-meta-llama-3-3-70b-instruct**). Use it to turn review highlights into a marketing tagline. Create a new SQL cell with the query below:

```
SELECT ai_query('databricks-meta-llama-3-3-70b-instruct','Based on these bakery customer reviews, write a 2-sentence marketing tagline in British English. Return only the tagline, no preamble. Reviews: ' ||array_join(collect_list(review),' ||| ')) AS tagline
FROM (
    SELECT review
    FROM samples.bakehouse.media_customer_reviews
    WHERE ai_classify(review,ARRAY('positive', 'negative', 'mixed', 'neutral')) = 'positive'
    LIMIT 10);
```

11. Run it and inspect the tagline. Re-run the cell a couple of times - the output varies because the model is non-deterministic. Try editing the prompt (e.g. "**write a formal tagline**" vs "**write a playful one**") to see how prompt wording shapes the result or swapping the foundational model to "**databricks-gpt-oss-120b**".
12. Combine everything in a view. Ask Genie Code:

> Using @samples.bakehouse.media_customer_reviews, create a view in the dbacademy.**<your schema>** schema called review_insights with these columns: the original review, a sentiment label from ai_classify, and a one-sentence summary from ai_summarize. Then show me the first 10 rows.

13. Review the DDL before running. AI functions in a view re-run on every query - fine for exploration, but in production you'd cache results in a table.

14. Finally, visualise the output. Ask Genie Code:

    > Generate a bar chart of sentiment label counts from my review_insights view.

15. Accept the chart and run it. Genie Code should make the chart available as a second tab in the cell result set beside "**Table**" - i.e. something like "**Sentiment Distribution**".

    You now have an end-to-end path from raw text to sentiment-labelled BI, all in SQL.

Lab 8: Genie Spaces - Talk to your Data

Objective: Create a Genie Space over a Unity Catalog table, curate it with a few business definitions, and have a natural-language conversation with your data - the same experience you would hand to a non-technical stakeholder.

Note: This lab only scratches the surface on what is possible in terms of setting up a Genie Space. For best results consult the Genie Spaces Best Practices documentation.

- In the left menu, click "**Genie**", then "**+ New**" to create a new Genie Space.
- In "**Connect your data**" type "**samples.nyctaxi.trips**" - select the table from the pick list, then click "**Create**".
- In the chat box (where it says "**ask your question …**"), toggle from "**Agent**" to "**Chat**".
- Now type:

  > How many trips are in this dataset and what date range do they cover?

- Review the generated answer. Click "**Show code**" (you may need to expand the result table and then click "**Show code**") to see the SQL Genie produced - this is how you verify Genie is doing the right thing.
- Ask something analytical:

  > What is the average fare amount per hour of day? Show the result as a bar chart.

- Follow up in the same chat to show conversational context:

  > Now show me the same thing but only for trips on weekends.

- Notice the shape - each question gets one SQL query, one answer, one chart. This is Chat mode: fast, focused, good for known questions.
- In the chat box toggle from "**Chat**" to "**Agent**":

> Weekend fares seemed different to weekday fares. Investigate why. Look at average fare, trip distance, trips per hour, and pickup location distribution. Tell me which factors most explain the weekend/weekday difference and where it shows up most strongly.

- Watch the thinking trace stream (click on "**Thinking …**") as Agent mode plans its approach, runs several queries in parallel, learns from each result, and iterates.
- Review the final report. You should see a structured summary with multiple supporting charts and tables, and citations back to the SQL Genie actually ran.
- Expand the citations to inspect any query you want to double-check.
- In summary:
  - **Chat** - one-shot SQL, immediate result, best for "what is X?" type questions
  - **Agent** - plan + multiple queries + synthesis, slower, best for "why is X happening?" or "find patterns in Y" type questions
- In the space settings, find "**Instructions**", "**General Instructions**" and paste:

> - A "long trip" is any trip with trip_distance greater than 10 miles.
>
> - Always exclude trips where fare_amount is 0 or negative - these are cancellations.
>
> - When grouping by location, use pickup_zip.

- Click "**Save**". Start a New chat in "**Chat**" mode and ask:

> What proportion of all trips are long trips, and which pickup zip has the most long trips?

- Confirm Genie applies your definitions without being told.
- Now ask a question Genie can't answer - same as before, but "really long" instead of "long".

> What proportion of all trips are really long trips?

- Genie should respond with something like "The question cannot be answered because "really long trip" is not defined. Please clarify what trip distance should be considered a "really long trip."
- Genie may also have go at defining it. Either way, provide the definition in the chat window as follows:

> a really long trip is more than 15 miles

- Genie will now run the query. But that definition only lives in this chat - add it to Instructions to make it available to everyone.
- Finally, add a Certified Question so a high-stakes answer is always consistent. In the space settings under "**SQL Queries**", click "**Add**".
  In "**What question does this query answer?**" type:

> What is the busiest hour of day by trip count?

In the SQL text box type:

```
SELECT hour(tpep_pickup_datetime) AS hour_of_day, count(*) AS trips
  FROM samples.nyctaxi.trips
  WHERE fare_amount > 0
  GROUP BY 1
  ORDER BY trips DESC
  LIMIT 24
```

Click "**Preview**" to make sure it runs and then "**Save**"

- Start a New chat in "**Chat**" mode and ask:

  > What is the busiest hour of day by trip count?

- Genie runs your certified SQL rather than generating fresh SQL (see this by clicking on "Show code").

What Next?

You have driven Genie Code through notebooks, ML, AI SQL functions, and a Genie Space. To keep building on what you have learned, here are some resources we recommend - all free.

Keep Practising in Your Own Workspace

Databricks Academy - if the company you work for is an existing Databricks customer, then you should have access to free self-paced courses via your work email address. Good starting points after this lab: *Generative AI Fundamentals*, *Get Started with Databricks for Data Analysis*, *Get Started with Databricks for Data Engineering*, and *Get Started with Databricks for Machine Learning*.

Go Deeper on the Topics in This Lab

- Genie Code Overview
- Genie Code overview, Use Genie Code, Genie Code for data science, Extend Genie Code with agent skills

- What is a Genie space, Curate an effective Genie space, Build a knowledge store, From Data to Dialogue (blog)
- AI functions on Databricks, ai_query reference

Watch and Learn

- The Start Learning courses in Free Edition (linked in the home page) - Databricks Fundamentals, Intro to Data Engineering and Intro to AI/BI
- **Databricks on YouTube** - product demos, conference talks, and how-to videos. Worth subscribing to for new feature drops.
- **Advancing Analytics** and **Bryan Cafferky** - two of the most respected independent Databricks channels for deeper engineering content.

Community and Events

- **Databricks Community** - ask questions, share notebooks, browse the Genie and AI/BI forums.
- **Data + AI Summit** - annual conference. Past sessions are free to stream and are a fast way to see what others are building with Genie, AI/BI, and Lakeflow.
- Join the **New Zealand Databricks User Group**